Лабораторна робота №2

Тема: Використання делегатів та подій у C#.

Завдання:

1. Написати програму для моделювання роботи банкомату
2. Передбачити можливості:
    - аутентифікація (перевірка введення номеру картки та пін-коду);
    - перегляд балансу на картці;
    - зняття коштів;
    - зарахування коштів на картку;
    - перерахування коштів на картку із заданим номером.

Повинна бути передбачена можливість додавання власних обробників подій на виконання операцій аутентифікації, перегляду балансу, зняття коштів та перерахування коштів на іншу картку.

Додайте власні обробники подій, які будуть виводити повідомлення *за допомогою виклику:*

- Console.WriteLine(“Повідомлення”);

При виконанні завдання створіть класи, які забезпечать роботу:

- Account - клас, що представляє картковий рахунок клієнта (властивості: номер картки, прізвище та ім’я власника, баланс на рахунку і т.д.);
- AutomatedTellerMachine - клас, що моделює роботу банкомата (містить кількість грошей, які містяться у банкоматі, ідентифікатор банкомата, адресу і т.п.);
- Bank - клас, що представляє банк (назва банку, список банкоматів і т.п.)

Це не повний список класів, за необхідності створюйте інші класи, які Вам будуть необхідні при реалізації завдання.

Лістинг коду класів з бібліотеки класів ClassLibraryATM:

Лістинг коду класу Account:

```
using ClassLibraryATM.Enums;

namespace ClassLibraryATM.Classes
{
    public class Account
    {
        public string? CardNumber { get; private set; }
        public string? OwnerFullName { get; private set; }
        public decimal Balance { get; private set; }
        private string? PinCode { get; set; }
        private bool IsBlocked { get; set; }
        private int FailedPinAttempts { get; set; }
        private DateTime ExpireDate { get; set; }
        private string? Currency { get; set; }
        public decimal DailyWithdrawLimit { get; private set; }
        public decimal WithdrawnToday { get; private set; }
        private DateTime LastWithdrawDate { get; set; }
        public List<Transaction> History { get; private set; }

        public AccountStatus Status;

        public Account() {
            CardNumber = "0000 0000 0000 0000";
            OwnerFullName = "Левченко Василіса";
            Balance = 0;
            PinCode = "0000";
            IsBlocked = false;
            FailedPinAttempts = 0;
            ExpireDate = new DateTime(2034, 03, 22);
            Currency = "USD";
            DailyWithdrawLimit = 100000m;
            WithdrawnToday = 0;
            LastWithdrawDate = new DateTime(2025, 11, 22);
            History = new List<Transaction>();
            Status = AccountStatus.Active;
        }

        public Account(string cardNumber, string ownerFullName, decimal balance, string
pinCode, string currency) : this()
        {
            if (cardNumber != null && cardNumber.Replace(" ", "").Length == 16 && card-
Number.All(char.IsDigit))
            {
                CardNumber = cardNumber;
            }
            OwnerFullName = ownerFullName;
            Balance = balance;
            if (pinCode != null && pinCode.Length == 4 && pinCode.All(char.IsDigit))
            {
                PinCode = pinCode;
            }
            if (currency != null && (currency.Length == 3 || currency.Length == 1))
            {
                Currency = currency;
            }
        }

        public Account(string cardNumber, string ownerFullName, decimal balance, string
pinCode, bool isBlocked, int failedPinAttempts, DateTime expireDate, string currency,
decimal dailyWithdrawLimit, decimal withdrawnToday, DateTime lastWithdrawDate,
List<Transaction> history, AccountStatus status) : this()
```

```
        {
            if (cardNumber != null && cardNumber.Replace(" ", "").Length == 16 && card-
Number.All(char.IsDigit))
            {
                CardNumber = cardNumber;
            }
            OwnerFullName = ownerFullName;
            Balance = balance;
            if (pinCode != null && pinCode.Length == 4 && pinCode.All(char.IsDigit))
            {
                PinCode = pinCode;
            }
            IsBlocked = isBlocked;
            FailedPinAttempts = failedPinAttempts;
            ExpireDate = expireDate;
            if (currency != null && (currency.Length == 3 || currency.Length == 1))
            {
                Currency = currency;
            }
            DailyWithdrawLimit = dailyWithdrawLimit;
            WithdrawnToday = withdrawnToday;
            LastWithdrawDate = lastWithdrawDate;
            History = history;
            Status = status;
        }

        public Account(Account other)
        {
            CardNumber = other.CardNumber;
            OwnerFullName = other.OwnerFullName;
            Balance = other.Balance;
            PinCode = other.PinCode;
            IsBlocked = other.IsBlocked;
            FailedPinAttempts = other.FailedPinAttempts;
            ExpireDate = other.ExpireDate;
            Currency = other.Currency;
            DailyWithdrawLimit = other.DailyWithdrawLimit;
            WithdrawnToday = other.WithdrawnToday;
            LastWithdrawDate = other.LastWithdrawDate;
            History = other.History != null ? new List<Transaction>(other.History) : new
List<Transaction>();
            Status = other.Status;
        }

        public bool VerifyPin(string pin)
        {
            if (IsBlocked) return false;

            if (pin.Length != 4 || !pin.All(char.IsDigit))
            {
                FailedPinAttempts++;
                if (FailedPinAttempts >= 3)
                {
                    IsBlocked = true;
                    Status = AccountStatus.Blocked;
                }
                return false;
            }

            if (pin != PinCode)
            {
                FailedPinAttempts++;
                if (FailedPinAttempts >= 3)
                {
                    IsBlocked = true;
```

```
                Status = AccountStatus.Blocked;
            }
            return false;
        }

        FailedPinAttempts = 0;
        return true;
    }

    public void Credit(decimal amount)
    {
        if (amount <= 0)
        {
            return;
        }

        if (amount > 29999m)
        {
            return;
        }

        Balance += amount;

        History.Add(new Transaction());
    }

    public void Debit(decimal amount)
    {
        if (amount <= 0)
        {
            return;
        }

        ResetDailyLimitIfNeeded();

        if (Balance < amount)
        {
            return;
        }

        if (WithdrawnToday + amount > DailyWithdrawLimit)
        {
            return;
        }

        WithdrawnToday += amount;
        Balance -= amount;

        History.Add(new Transaction());
    }

    public void ResetDailyLimitIfNeeded()
    {
        if (LastWithdrawDate.Date != DateTime.Today)
        {
            WithdrawnToday = 0;
            LastWithdrawDate = DateTime.Today;
        }
    }
  }
}
```

Лістинг коду класу AutomatedTellerMachine:

```
using ClassLibraryATM.Enums;

namespace ClassLibraryATM.Classes
{
    public class AutomatedTellerMachine
    {
        public string? AtmId { get; private set; }
        public string? Address { get; private set; }
        public decimal CashAvailable { get; set; }
        private bool IsOnline { get; set; }
        public Bank OwnerBank { get; private set; }
        private DateTime LastServiceDate { get; set; }
        private decimal MaxWithdrawPerOperation { get; set; }
        private decimal FeePercent { get; set; }
        public List<Transaction> AtmJournal { get; private set; }

        public Account? CurrentAccount { get; private set; }

        public AtmState State;

        public AutomatedTellerMachine() {
            AtmId = "0";
            Address = "Вулиця Героїв Чорнобилю";
            CashAvailable = 0;
            IsOnline = true;
            OwnerBank = new Bank();
            LastServiceDate = new DateTime(2025, 09, 18, 22, 32, 24);
            MaxWithdrawPerOperation = 0;
            FeePercent = 0;
            AtmJournal = new List<Transaction>();
            CurrentAccount = new Account();
            State = AtmState.Authenticated;
        }

        public AutomatedTellerMachine(string atmId, string address, decimal
cashAvailable, bool isOnline, Bank ownerBank)
        {
            AtmId = atmId;
            Address = address;
            CashAvailable = cashAvailable;
            IsOnline = isOnline;
            OwnerBank = ownerBank;
            LastServiceDate = new DateTime(2025, 09, 20, 18, 12, 34);
            MaxWithdrawPerOperation = 0;
            FeePercent = 0;
            AtmJournal = new List<Transaction>();
            CurrentAccount = new Account();
            State = AtmState.Authenticated;
        }

        public AutomatedTellerMachine(Bank ownerBank)
        {
            OwnerBank = ownerBank;
            IsOnline = true;
            AtmJournal = new List<Transaction>();
            CurrentAccount = new Account();
            State = AtmState.Authenticated;
            LastServiceDate = DateTime.Now;
        }

        public AutomatedTellerMachine(string atmId, string address, decimal
cashAvailable, bool isOnline, Bank ownerBank, DateTime lastServiceDate, decimal
```

```
maxWithdrawPerOperation, decimal feePercent, List<Transaction> atmJournal, Account
currentAccount, AtmState state)
        {
            AtmId = atmId;
            Address = address;
            CashAvailable = cashAvailable;
            IsOnline = isOnline;
            OwnerBank = ownerBank;
            LastServiceDate = lastServiceDate;
            MaxWithdrawPerOperation = maxWithdrawPerOperation;
            FeePercent = feePercent;
            AtmJournal = atmJournal;
            CurrentAccount = currentAccount;
            State = state;
        }

        public AutomatedTellerMachine(AutomatedTellerMachine other)
        {
            AtmId = other.AtmId;
            Address = other.Address;
            CashAvailable = other.CashAvailable;
            IsOnline = other.IsOnline;
            OwnerBank = other.OwnerBank != null ? new Bank(other.OwnerBank) : new Bank();
            LastServiceDate = other.LastServiceDate;
            MaxWithdrawPerOperation = other.MaxWithdrawPerOperation;
            FeePercent = other.FeePercent;
            AtmJournal = other.AtmJournal != null ? new
List<Transaction>(other.AtmJournal) : new List<Transaction>();
            CurrentAccount = other.CurrentAccount != null ? new
Account(other.CurrentAccount) : new Account();
            State = other.State;
        }

        public void Logout()
        {
            CurrentAccount = null;
            State = AtmState.CardInserted;
        }

        public delegate void AuthEventHandler(string cardNumber, bool success, string
message);
        public event AuthEventHandler? Authenticated;

        public delegate void BalanceEventHandler(Account? acc, decimal balance, string
msg);
        public event BalanceEventHandler? BalanceChecked;

        public delegate void WithdrawEventHandler(Account? acc, decimal amount, bool
success, string msg);
        public event WithdrawEventHandler? WithdrawCompleted;

        public delegate void DepositEventHandler(Account? acc, decimal amount, bool
success, string msg);
        public event DepositEventHandler? DepositCompleted;

        public delegate void TransferEventHandler(Account? from, Account? to, decimal
amount, bool success, string msg);
        public event TransferEventHandler? TransferCompleted;

        public bool Authenticate(string cardNumber, string pin)
        {
            if (!IsOnline)
            {
                State = AtmState.OutOfService;
                Authenticated?.Invoke(cardNumber, false, "Банкомат не активний.");
```

```
                return false;
            }

            if (string.IsNullOrWhiteSpace(cardNumber) || string.IsNullOrWhiteSpace(pin))
            {
                State = AtmState.CardInserted;
                Authenticated?.Invoke(cardNumber ?? "NULL", false, "Некоректні дані для
авторизації.");
                return false;
            }

            var account = OwnerBank.FindAccount(cardNumber);

            if (account == null)
            {
                Authenticated?.Invoke(cardNumber, false, "Картку не знайдено.");
                return false;
            }

            bool ok = account.VerifyPin(pin);

            if (ok)
            {
                CurrentAccount = account;
                State = AtmState.Authenticated;
                Authenticated?.Invoke(cardNumber, true, "PIN код успішно підтверджено.");
                return true;
            }
            else if (!ok)
            {
                State = AtmState.CardInserted;
                Authenticated?.Invoke(cardNumber, false, "Некоректний PIN код.");
                return false;
            }
            else
            {
                State = AtmState.Locked;
                Authenticated?.Invoke(cardNumber, false, "Ви більше не можете вводити
операції вводу для авторизації.");
                return false;
            }
        }

        public void CheckBalance()
        {
            if (CurrentAccount == null)
            {
                BalanceChecked?.Invoke(null, 0, "Немає активного рахунку.");
                return;
            }

            if (State != AtmState.Authenticated)
            {
                BalanceChecked?.Invoke(CurrentAccount, 0, "Користувач не авторизова-
ний.");
                return;
            }

            if (IsMaintenanceExpired())
            {
                State = AtmState.OutOfService;
                BalanceChecked?.Invoke(CurrentAccount, CurrentAccount.Balance, "Банкомат
потребує обслуговування.");
                return;
            }
```

```
            BalanceChecked?.Invoke(CurrentAccount, CurrentAccount.Balance, "Баланс успіш-
но отримано.");
        }

        public void Withdraw(decimal amount)
        {
            if (CurrentAccount == null)
            {
                WithdrawCompleted?.Invoke(null!, amount, false, "Немає активного рахун-
ку.");
                return;
            }

            if (!IsOnline)
            {
                State = AtmState.OutOfService;
                WithdrawCompleted?.Invoke(CurrentAccount, amount, false, "Банкомат не
активний.");
                return;
            }

            if (State != AtmState.Authenticated)
            {
                WithdrawCompleted?.Invoke(CurrentAccount, amount, false, "Користувач не
авторизований.");
                return;
            }

            if (CurrentAccount.Status != AccountStatus.Active)
            {
                State = AtmState.CardInserted;
                WithdrawCompleted?.Invoke(CurrentAccount, amount, false, "Картка не акти-
вна.");
                return;
            }

            if (amount <= 0)
            {
                State = AtmState.Authenticated;
                WithdrawCompleted?.Invoke(CurrentAccount, amount, false, "Некоректна су-
ма.");
                return;
            }

            if (CashAvailable < amount)
            {
                State = AtmState.Authenticated;
                WithdrawCompleted?.Invoke(CurrentAccount, amount, false, "У банкоматі
недостатньо коштів.");
                return;
            }

            if (CurrentAccount.Balance < amount)
            {
                State = AtmState.Authenticated;
                WithdrawCompleted?.Invoke(CurrentAccount, amount, false, "На рахунку не-
достатньо коштів.");
                return;
            }

            if (amount > CurrentAccount.DailyWithdrawLimit)
            {
                State = AtmState.Authenticated;
```

```
                WithdrawCompleted?.Invoke(CurrentAccount, amount, false, "Сума перевищує
денний ліміт.");
                return;
            }

            if (amount > MaxWithdrawPerOperation)
            {
                State = AtmState.Authenticated;
                WithdrawCompleted?.Invoke(CurrentAccount, amount, false, "Сума перевищує
ліміт на одну операцію.");
                return;
            }

            decimal fee = amount * FeePercent / 100;
            decimal total = amount + fee;

            if (CurrentAccount.Balance < total)
            {
                State = AtmState.Authenticated;
                WithdrawCompleted?.Invoke(CurrentAccount, amount, false, "Недостатньо
коштів з урахуванням комісії.");
                return;
            }

            if (IsMaintenanceExpired())
            {
                State = AtmState.OutOfService;
                WithdrawCompleted?.Invoke(CurrentAccount, amount, false, "Банкомат потре-
буе обслуговування.");
                return;
            }

            var transaction = new Transaction(
                TransactionType.Widthdraw,
                amount,
                CurrentAccount.CardNumber ?? string.Empty,
                "",
                DateTime.Now,
                fee
                );

            CurrentAccount.Debit(total);
            CashAvailable -= amount;

            AtmJournal.Add(transaction);
            CurrentAccount.History.Add(transaction);

            WithdrawCompleted?.Invoke(CurrentAccount, amount, true, "Зняття готівки вико-
нано.");
        }

        public void Deposit(decimal amount)
        {
            if (CurrentAccount == null)
            {
                DepositCompleted?.Invoke(null!, amount, false, "Немає активного рахун-
ку.");
                return;
            }

            if (!IsOnline)
            {
                State = AtmState.OutOfService;
                DepositCompleted?.Invoke(CurrentAccount, amount, false, "Банкомат не ак-
тивний.");
```

```
                return;
            }

            if (State != AtmState.Authenticated)
            {
                DepositCompleted?.Invoke(CurrentAccount, amount, false, "Користувач не
авторизований.");
                return;
            }

            if (CurrentAccount.Status != AccountStatus.Active)
            {
                State = AtmState.Authenticated;
                DepositCompleted?.Invoke(CurrentAccount, amount, false, "Картка не актив-
на.");
                return;
            }

            if (amount <= 0)
            {
                State = AtmState.Authenticated;
                DepositCompleted?.Invoke(CurrentAccount, amount, false, "Некоректна су-
ма.");
                return;
            }

            if (IsMaintenanceExpired())
            {
                State = AtmState.OutOfService;
                DepositCompleted?.Invoke(CurrentAccount, amount, false, "Банкомат потре-
бує обслуговування.");
                return;
            }

            CurrentAccount.Credit(amount);
            CashAvailable += amount;

            DepositCompleted?.Invoke(CurrentAccount, amount, true, "Зарахування готівки
відбулося успішно.");
        }

        public void Transfer(string toCardNumber, decimal amount)
        {
            if (CurrentAccount == null)
            {
                TransferCompleted?.Invoke(null, null, amount, false, "Немає активного
рахунку.");
                return;
            }

            if (!IsOnline)
            {
                State = AtmState.OutOfService;
                TransferCompleted?.Invoke(CurrentAccount, CurrentAccount, amount, false,
"Банкомат не активний.");
                return;
            }

            if (CurrentAccount.Status != AccountStatus.Active)
            {
                State = AtmState.Authenticated;
                TransferCompleted?.Invoke(CurrentAccount, CurrentAccount, amount, false,
"Картка не активна.");
                return;
            }
```

```
            if (amount <= 0)
            {
                State = AtmState.Authenticated;
                TransferCompleted?.Invoke(CurrentAccount, CurrentAccount, amount, false,
"Некоректна сума.");
                return;
            }

            var toAccount = OwnerBank.FindAccount(toCardNumber);
            if (toAccount == null)
            {
                State = AtmState.Authenticated;
                TransferCompleted?.Invoke(CurrentAccount, null, amount, false, "Рахунок
одержувача не знайдено.");
                return;
            }

            if (CurrentAccount.Balance < amount)
            {
                State = AtmState.Authenticated;
                TransferCompleted?.Invoke(CurrentAccount, toAccount, amount, false, "На
рахунку недостатньо коштів.");
                return;
            }

            if (CurrentAccount.WithdrawnToday + amount >
CurrentAccount.DailyWithdrawLimit)
            {
                State = AtmState.Authenticated;
                TransferCompleted?.Invoke(CurrentAccount, toAccount, amount, false, "Сума
перевищує денний ліміт.");
                return;
            }

            decimal fee = amount * FeePercent / 100;
            decimal total = amount + fee;

            if (CurrentAccount.Balance < total)
            {
                State = AtmState.Authenticated;
                TransferCompleted?.Invoke(CurrentAccount, toAccount, amount, false, "Не-
достатньо коштів для переказу коштів з урахуванням комісії.");
                return;
            }

            if (IsMaintenanceExpired())
            {
                State = AtmState.OutOfService;
                TransferCompleted?.Invoke(CurrentAccount, toAccount, amount, false, "Бан-
комат потребує обслуговування.");
                return;
            }

            var trasnsaction = new Transaction(
                TransactionType.Transfer,
                amount,
                CurrentAccount.CardNumber ?? string.Empty,
                toAccount.CardNumber ?? string.Empty,
                DateTime.Now,
                fee
                );

            CurrentAccount.Debit(total);
            toAccount.Credit(amount);
```

```
            AtmJournal.Add(trasnsaction);
            CurrentAccount.History.Add(trasnsaction);
            toAccount.History.Add(trasnsaction);

            TransferCompleted?.Invoke(CurrentAccount, toAccount, amount, true, "Перехау-
вання виконано.");
        }

        private bool IsMaintenanceExpired()
        {
            return (DateTime.Now - LastServiceDate).TotalDays > 180;
        }
    }
}
```

Лістинг коду класу Bank:

```
namespace ClassLibraryATM.Classes
{
    public class Bank
    {
        public string? Name { get; private set; }
        public List<AutomatedTellerMachine>? Atms { get; private set; }
        public Dictionary<string, Account> Accounts { get; private set; }
        private decimal TransferFeePercent { get; set; }
        private decimal MinBalanceRequired { get; set; }
        public List<Transaction> BackLedger { get; private set; }

        public Bank()
        {
            Name = "ATM №12";
            Atms = new List<AutomatedTellerMachine>();
            Accounts = new Dictionary<string, Account>();
            TransferFeePercent = 0;
            MinBalanceRequired = 0;
            BackLedger = new List<Transaction>();
        }

        public Bank(string name, List<AutomatedTellerMachine> atms, Dictionary<string,
Account> accounts) : this()
        {
            Name = name;
            Atms = atms;
            Accounts = accounts;
        }

        public Bank(string name, List<AutomatedTellerMachine> atms, Dictionary<string,
Account> accounts, decimal transferFeePercent, decimal minBalanceRequired,
List<Transaction> backLedger) : this()
        {
            Name = name;
            Atms = atms;
            Accounts = accounts;
            TransferFeePercent = transferFeePercent;
            MinBalanceRequired = minBalanceRequired;
            BackLedger = backLedger;
        }

        public Bank(Bank other)
        {
            Name = other.Name;
            Atms = other.Atms != null ? new List<AutomatedTellerMachine>(other.Atms) :
new List<AutomatedTellerMachine>();
```

```
            Accounts = other.Accounts != null ? new Dictionary<string, Ac-
count>(other.Accounts) : new Dictionary<string, Account>();
            TransferFeePercent = other.TransferFeePercent;
            MinBalanceRequired = other.MinBalanceRequired;
            BackLedger = other.BackLedger != null ? new
List<Transaction>(other.BackLedger) : new List<Transaction>();
        }

        public void RegisterAccount(Account acc)
        {
            if (acc.CardNumber != null)
            {
                Accounts[acc.CardNumber] = acc;
            }
        }

        public Account? FindAccount(string cardNumber)
        {
            Accounts.TryGetValue(cardNumber, out var acc);
            return acc;
        }
    }
}
```

Лістинг коду класу Transaction:

```
using ClassLibraryATM.Enums;

namespace ClassLibraryATM.Classes
{
    public class Transaction
    {
        public TransactionType Type { get; private set; }
        public decimal Amount { get; private set; }
        public string FromCard { get; private set; }
        public string ToCard { get; private set; }
        public DateTime Date { get; private set; }

        public decimal Fee { get; private set; }

        public Transaction()
        {
            Type = TransactionType.CheckBalance;
            Amount = 0;
            FromCard = "0000 0000 0000 0000";
            ToCard = "0000 0000 0000 0000";
            Date = new DateTime(2025, 10, 23);
            Fee = 0;
        }

        public Transaction(TransactionType type, decimal amount, string fromCard, string
toCard) : this()
        {
            Type = type;
            Amount = amount;
            FromCard = fromCard;
            ToCard = toCard;
        }

        public Transaction(TransactionType type, decimal amount, string fromCard, string
toCard, DateTime date, decimal fee) : this()
        {
            Type = type;
            Amount = amount;
```

```
            FromCard = fromCard;
            ToCard = toCard;
            Date = date;
            Fee = fee;
        }

        public Transaction(Transaction other)
        {
            Type = other.Type;
            Amount = other.Amount;
            FromCard = other.FromCard;
            ToCard = other.ToCard;
            Date = other.Date;
            Fee = other.Fee;
        }
    }
}
```

Лістинг коду енумераторів з бібліотеки класів ClassLibraryATM:

Лістинг коду енумератора AccountStatus:

```
namespace ClassLibraryATM.Enums
{
    public enum AccountStatus
    {
        Active,
        Blocked,
        Expired
    }
}
```

Лістинг коду енуметратора AtmState:

```
namespace ClassLibraryATM.Enums
{
    public enum AtmState
    {
        Idle,
        CardInserted,
        Authenticated,
        OutOfService,
        Locked
    }
}
```

Лістинг коду енумератора TransactionType:

```
namespace ClassLibraryATM.Enums
{
    public enum TransactionType
    {
        Widthdraw,
        Deposit,
        Transfer,
        CheckBalance
    }
}
```

Лістинг коду класу Program з перевіркою роботоздатності програми:

```
using System.Text;
using ClassLibraryATM.Classes;
using ClassLibraryATM.Enums;

class Program
{
    static void Main()
    {
        Console.InputEncoding = Encoding.UTF8;
        Console.OutputEncoding = Encoding.UTF8;

        Console.WriteLine("------------------------------------------------------------
");
        Console.WriteLine("Лабораторну робота №2\nВиконав:\nстудент: Черепанов
І.І.\nгрупа:ЗІПЗ-24-1");
        Console.WriteLine("------------------------------------------------------------
");
        Console.WriteLine("Ласкаво просимо у банкомат!");

        var bank = new Bank();

        var account1 = new Account("3456234556784567", "Черепанов Ілля", 0, "3451",
false, 0, new DateTime(2028, 09, 15), "UAH", 100000m, 0, new DateTime(2025, 11, 23), new
List<Transaction>(), AccountStatus.Active);
        var account2 = new Account("2345547434526786", "Левченко Крістіна", 0, "4655",
false, 0, new DateTime(2028, 10, 12), "UAH", 100000m, 0, new DateTime(2025, 10, 26), new
List<Transaction>(), AccountStatus.Active);

        bank.RegisterAccount(account1);
        bank.RegisterAccount(account2);

        var ATM = new AutomatedTellerMachine(bank);

        ATM.Authenticated += (card, success, msg) =>
        {
            Console.WriteLine($"Подія авторизації => {msg} (success = {success})");

            if (!success)
            {
                Console.WriteLine("Авторизація неуспішна! Спробуйте ще раз.\n");
            }
            else
            {
                Console.WriteLine("Успішна авторизація");
            }
        };

        ATM.BalanceChecked += (acc, bal, msg) =>
        {
            Console.WriteLine(msg);
            Console.WriteLine($"Баланс: {bal}");
        };

        ATM.WithdrawCompleted += (acc, amt, ok, msg) =>
        {
            Console.WriteLine(msg);
            if (ok) Console.WriteLine($"Знято: {amt}");
        };

        ATM.DepositCompleted += (acc, amt, ok, msg) =>
        {
            Console.WriteLine(msg);
            if (ok) Console.WriteLine($"Поповнено: {amt}");
```

```
        };

        ATM.TransferCompleted += (from, to, amt, ok, msg) =>
        {
            Console.WriteLine(msg);
            if (ok) Console.WriteLine($"Переказано {amt} на {to?.CardNumber}");
        };

        bool execute = true;

        while (execute)
        {
            Console.WriteLine("(Для виходу, введіть 'exit')");
            Console.Write("Номер картки: ");
            string? card = Console.ReadLine();
            Console.Write("PIN: ");
            string? pin = Console.ReadLine();

            if (card?.ToLower() == "exit" || pin?.ToLower() == "exit")
            {
                Console.WriteLine("\n-----------------------------------------------------
----------");
                Console.WriteLine("Завершення роботи програм. Формування звіту...");

                Console.WriteLine("\n=== Звіт про транзакції ===");

                foreach (var acc in bank.Accounts)
                {
                    Console.WriteLine($"\nКартка: {acc.Key} -
{acc.Value.OwnerFullName}");
                    if (acc.Value.History.Count == 0)
                    {
                        Console.WriteLine("  (Транзакцій немає)");
                        continue;
                    }

                    foreach (var tr in acc.Value.History)
                    {
                        Console.WriteLine($"  {tr.Date}: {tr.Type} - {tr.Amount} UAH");
                    }
                }

                Console.WriteLine($"\n=== Стан банкомата ===");
                Console.WriteLine($"Кількість грошей у наявності: {ATM.CashAvailable}");
                Console.WriteLine($"Стан: {ATM.State}");
                Console.WriteLine("-----------------------------------------------------
--------");

                execute = false;
                break;
            }

            if (string.IsNullOrWhiteSpace(card) || string.IsNullOrWhiteSpace(pin))
            {
                Console.WriteLine("Номер картки та PIN не можуть бути порожніми.
Спробуйте ще раз!\n");
                continue;
            }

            bool success = ATM.Authenticate(card, pin);

            if (!success)
            {
                Console.WriteLine("Спробуйте ще раз!\n");
                continue;
```

```
            }

            bool exitMenu = false;
            while (!exitMenu)
            {
                Console.WriteLine("\nМеню:");
                Console.WriteLine("1. Баланс");
                Console.WriteLine("2. Зняти кошти");
                Console.WriteLine("3. Поповнити картку");
                Console.WriteLine("4. Переказати кошти");
                Console.WriteLine("5. Вийти");

                if (!int.TryParse(Console.ReadLine(), out int choice))
                {
                    Console.WriteLine("Некоректний вибір");
                    continue;
                }

                switch (choice)
                {
                    case 1:
                        ATM.CheckBalance();
                        break;

                    case 2:
                        Console.Write("Сума для зняття: ");
                        decimal w = decimal.Parse(Console.ReadLine()!);
                        ATM.Withdraw(w);
                        break;

                    case 3:
                        Console.Write("Сума для поповнення: ");
                        decimal d = decimal.Parse(Console.ReadLine()!);
                        ATM.Deposit(d);
                        break;

                    case 4:
                        Console.Write("Картка отримувача: ");
                        string dest = Console.ReadLine()!;
                        Console.Write("Сума переказу: ");
                        decimal t = decimal.Parse(Console.ReadLine()!);
                        ATM.Transfer(dest, t);
                        break;

                    case 5:
                        exitMenu = true;
                        ATM.Logout();
                        break;

                    default:
                        Console.WriteLine("Виберіть пункт від 1 до 5");
                        break;
                }
            }
        }
    }
}
```

Результат виконання програми:

```
---------------------------------------------------------------
Лабораторну робота №2
Виконав:
студент: Черепанов І.І.
група:ЗІПЗ-24-1
---------------------------------------------------------------
Ласкаво просимо у банкомат!
(Для виходу, введіть 'exit')
Номер картки: 3456234556784567
PIN: 3451
Подія авторизації => PIN код успішно підтверджено. (success = True)
Успішна авторизація

Меню:
1. Баланс
2. Зняти кошти
3. Поповнити картку
4. Переказати кошти
5. Вийти
1
Баланс успішно отримано.
Баланс: 0

Меню:
1. Баланс
2. Зняти кошти
3. Поповнити картку
4. Переказати кошти
5. Вийти
3
Сума для поповнення: 1200
Зарахування готівки відбулося успішно.
Поповнено: 1200

Меню:
1. Баланс
2. Зняти кошти
3. Поповнити картку
4. Переказати кошти
5. Вийти
2
Сума для зняття: 200
Зняття готівки виконано.
Знято: 200

Меню:
1. Баланс
2. Зняти кошти
3. Поповнити картку
4. Переказати кошти
5. Вийти
1
Баланс успішно отримано.
Баланс: 1000

Меню:
1. Баланс
2. Зняти кошти
3. Поповнити картку
4. Переказати кошти
```

```
5. Вийти
4
Картка отримувача: 2345547434526786
Сума переказу: 300
Перехаування виконано.
Переказано 300 на 2345547434526786

Меню:
1. Баланс
2. Зняти кошти
3. Поповнити картку
4. Переказати кошти
5. Вийти
1
Баланс успішно отримано.
Баланс: 700

Меню:
1. Баланс
2. Зняти кошти
3. Поповнити картку
4. Переказати кошти
5. Вийти
5
(Для виходу, введіть 'exit')
Номер картки: 2345547434526786
PIN: 4655
Подія авторизації => PIN код успішно підтверджено. (success = True)
Успішна авторизація

Меню:
1. Баланс
2. Зняти кошти
3. Поповнити картку
4. Переказати кошти
5. Вийти
1
Баланс успішно отримано.
Баланс: 300

Меню:
1. Баланс
2. Зняти кошти
3. Поповнити картку
4. Переказати кошти
5. Вийти
2
Сума для зняття: 300
Зняття готівки виконано.
Знято: 300

Меню:
1. Баланс
2. Зняти кошти
3. Поповнити картку
4. Переказати кошти
5. Вийти
1
Баланс успішно отримано.
Баланс: 0
```

```
Меню:
1. Баланс
2. Зняти кошти
3. Поповнити картку
4. Переказати кошти
5. Вийти
5
(Для виходу, введіть 'exit')
Номер картки: exit
PIN: exit

----------------------------------------------------------------
Завершення роботи програм. Формування звіту...

=== Звіт про транзакції ===

Картка: 3456234556784567 - Черепанов Ілля
  23/10/2025 00:00:00: CheckBalance - 0 UAH
  23/10/2025 00:00:00: CheckBalance - 0 UAH
  25/11/2025 20:02:54: Widthdraw - 200 UAH
  23/10/2025 00:00:00: CheckBalance - 0 UAH
  25/11/2025 20:03:17: Transfer - 300 UAH

Картка: 2345547434526786 - Левченко Крістіна
  23/10/2025 00:00:00: CheckBalance - 0 UAH
  25/11/2025 20:03:17: Transfer - 300 UAH
  23/10/2025 00:00:00: CheckBalance - 0 UAH
  25/11/2025 20:04:04: Widthdraw - 300 UAH

=== Стан банкомата ===
Кількість грошей у наявності: 700
Стан: CardInserted
----------------------------------------------------------------
```

Рис. 1, 2, 3. Результат виконання програми

Посилання на репозиторій проекту

Висновки: Виконуючи лабораторну роботу №2 я зміг засвоїти знання та практику використання делегатів та подій у мові C#.